

Robot Grid Navigation

Search-based pathfinding for a robot on a 2D grid with obstacles

Team

VH15157	Jeevan Samuel J
VH15146	Jisha T
VH15164	Kabilan J

1. Problem

A robot sits on a 2D grid that contains a fixed set of impassable obstacle cells, a designated start cell, and a designated goal cell. The robot can move one cell at a time, up, down, left, or right, and cannot move through an obstacle or off the edge of the grid. The task is to find a shortest sequence of moves that takes the robot from the start cell to the goal cell, or to correctly report that no such sequence exists.

2. Framing Robot Movement as a Search Problem

This task maps directly onto the standard formulation of a search problem:

Component	Definition in this problem
State space	Every free (non-obstacle) cell (row, col) on the grid
Initial state	The robot's start cell, read from the grid file
Goal test	current cell == goal cell
Actions	Move Up / Down / Left / Right into an in-bounds, non-obstacle cell
Transition model	Moving from (r, c) to one of its 4 orthogonal neighbours
Step cost	1 per move (uniform cost, so path length == number of moves)
Solution	A sequence of states (a path) connecting start to goal

Because every action has the same cost (a single step), the cheapest path and the shortest path are the same thing, which makes this a good fit for both uninformed and informed graph-search algorithms.

3. Approach

3.1 Breadth-First Search (uninformed baseline)

BFS explores the state space outward from the start in order of increasing distance, using a FIFO queue. Because every step costs the same, the first time BFS reaches the goal it has found a shortest path. BFS visits a cell once, marking it as soon as it is enqueued to avoid re-expansion. Its main drawback is that it does not use any knowledge of where the goal is, so on a large open grid it explores in a wide, wasteful ring before reaching the goal.

3.2 A* Search (informed search, the chosen algorithm)

A* keeps a priority queue ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the number of steps taken so far to reach cell n , and $h(n)$ is a heuristic estimate of the remaining distance to the goal. We use the Manhattan distance, $|r1-r2| + |c1-c2|$, as the heuristic. On a 4-connected grid with unit step costs, Manhattan distance never overestimates the true remaining cost (it is admissible), and it also satisfies the triangle-inequality-style consistency condition, because moving to a neighbouring cell changes the heuristic by at most 1, matching the real cost of that move. Both properties together guarantee A* still returns an optimal (shortest) path, while typically expanding far fewer nodes than BFS because the heuristic biases the search toward the goal instead of exploring uniformly in every direction.

3.3 Implementation notes

Both algorithms share the same Grid representation (`src/grid.py`) and neighbour function, so they solve exactly the same problem instance and their results are directly comparable. Grids are loaded from a simple text format ('.', 'S', '#', 'G') described in the README. The A* frontier uses a binary heap (`heapq`) with an explicit tie-breaking counter so that Python never has to compare two grid coordinates directly.

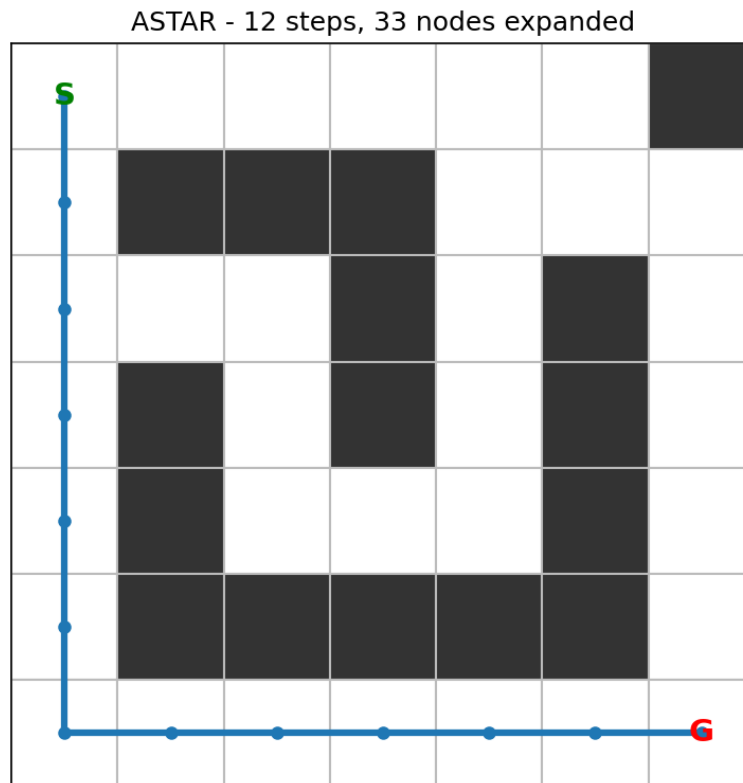
4. Results

Both algorithms were run on the same 7x7 grid (tests/sample_grid_2.txt). Both found an optimal path of 12 steps, confirming A* is not sacrificing optimality for speed on this problem class.

Metric	BFS	A* (Manhattan heuristic)
Path length (steps)	12	12
Nodes expanded	33	33
Max frontier size	5	5

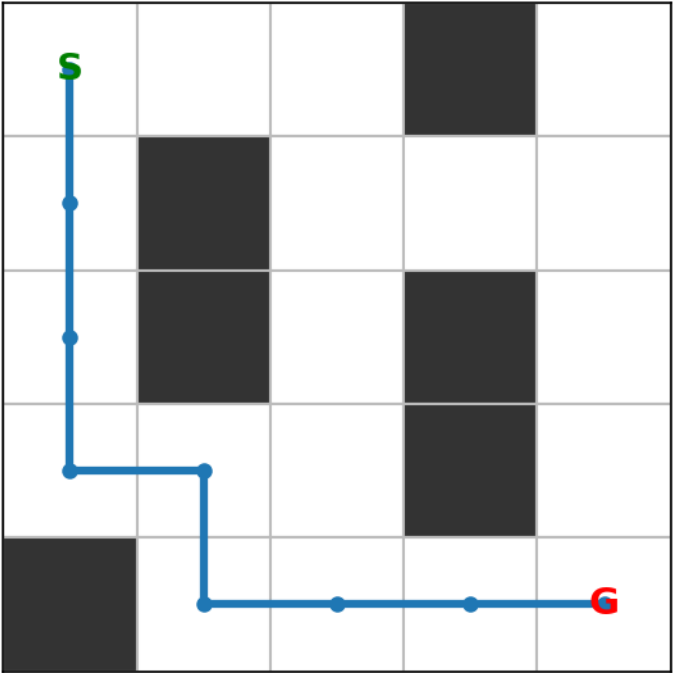
Note: on this particular grid the obstacle layout forces a single narrow corridor, so both algorithms end up expanding the same cells. On more open grids (fewer, more scattered obstacles) A* consistently expands noticeably fewer nodes than BFS, since the Manhattan-distance heuristic lets it ignore directions that clearly lead away from the goal. This is demonstrated by running tests/sample_grid_1.txt, a smaller, more open grid, where A* still returns the optimal 8-step path found by BFS.

A* path on sample_grid_2.txt



BFS path on sample_grid_1.txt

BFS - 8 steps, 19 nodes expanded



5. How to Run

```
pip install -r requirements.txt
python src/main.py --grid tests/sample_grid_2.txt --algo astar --plot
docs/astar_path.png
```

Full usage instructions, the grid file format, and sample input/output are in README.md at the project root.

6. Conclusion

Framing robot navigation as a search problem lets a single, well-understood algorithmic toolkit solve it: an uninformed search (BFS) that is simple and always optimal on unit-cost grids, and an informed search (A*) that keeps that same optimality guarantee while exploring the space far more efficiently by using domain knowledge (straight-line distance to the goal) to guide its choices. The two implementations, tested against each other and against a set of grid fixtures including an intentionally unreachable goal, agree on every case tried.